

MVC / MVP – RAFFINARE

Abbiamo analizzato il BCE, un pattern di analisi per i sistemi object-oriented.

Il suo obiettivo principale è individuare le **macro-classi** del sistema, derivate dall'osservazione del dominio e dai modi in cui il sistema viene utilizzato dagli attori (cioè dai **casi d'uso**).

A questo punto sorge una domanda:

come è possibile **raffinare un modello di analisi BCE** per ottenere un **diagramma delle classi** che sia più vicino a quello che sarà poi l'effettiva implementazione del sistema?

In altre parole:

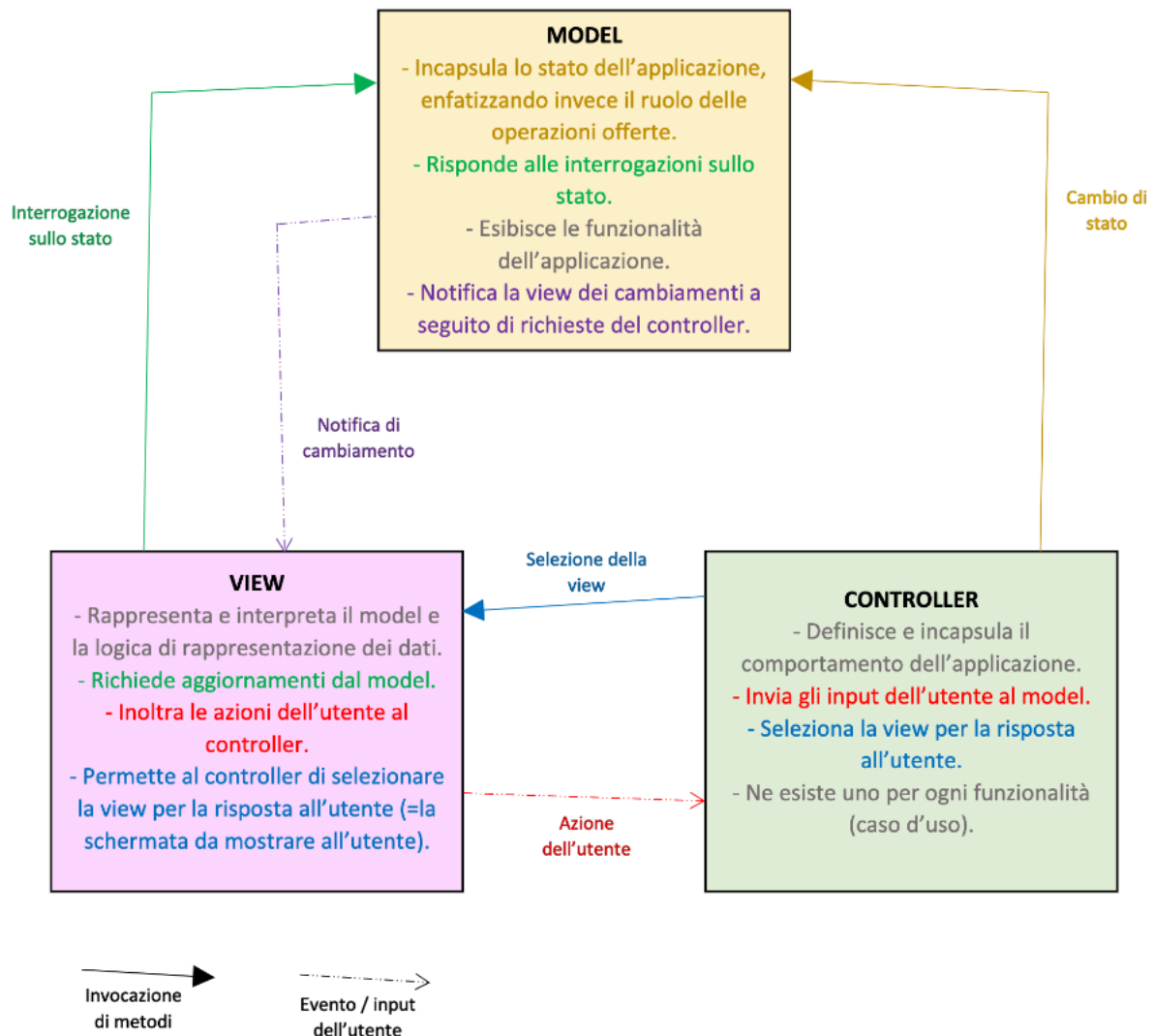
come si possono introdurre e gestire, nel dominio della soluzione, **aspetti legati all'ingegnerizzazione del sistema**, come il **riuso del codice**, l'**ottimizzazione delle soluzioni**, la **manutenibilità** e la **scalabilità**?

La risposta è che il modello di analisi basato su BCE può essere **raffinato secondo diverse varianti architetturali**, tra cui le più importanti sono:

- **Model View Controller (MVC)**
- **Model View Presenter (MVP)**

Ci siamo spostati nel dominio della soluzione.

MVC – ANALISI IN DETTAGLIO



Abbiamo gli artefatti di analisi BCE, che vengono successivamente **raffinati in artefatti di progettazione** secondo lo schema MVC.

La soluzione basata su MVC deriva dunque direttamente dalla soluzione ottenuta con il BCE.

Anche nell'architettura MVC ritroviamo tre livelli distinti: **Model**, **View** e **Controller**.

La *View* è responsabile di:

- gestire la **logica di presentazione dei dati**;
- occuparsi della **costruzione e gestione dell'interfaccia grafica (GUI)**;
- curare l'**acquisizione dei dati** da parte dell'applicazione;

- consentire l'accesso in sola lettura ai dati presenti nel *Model*.

Inoltre, qualora non se ne occupi il *Controller*, la *View* può anche gestire la **conversione dei dati** tra il **formato esterno** (cioè come appaiono all'utente) e il **formato interno** (cioè come sono memorizzati nel *Model*), e viceversa.

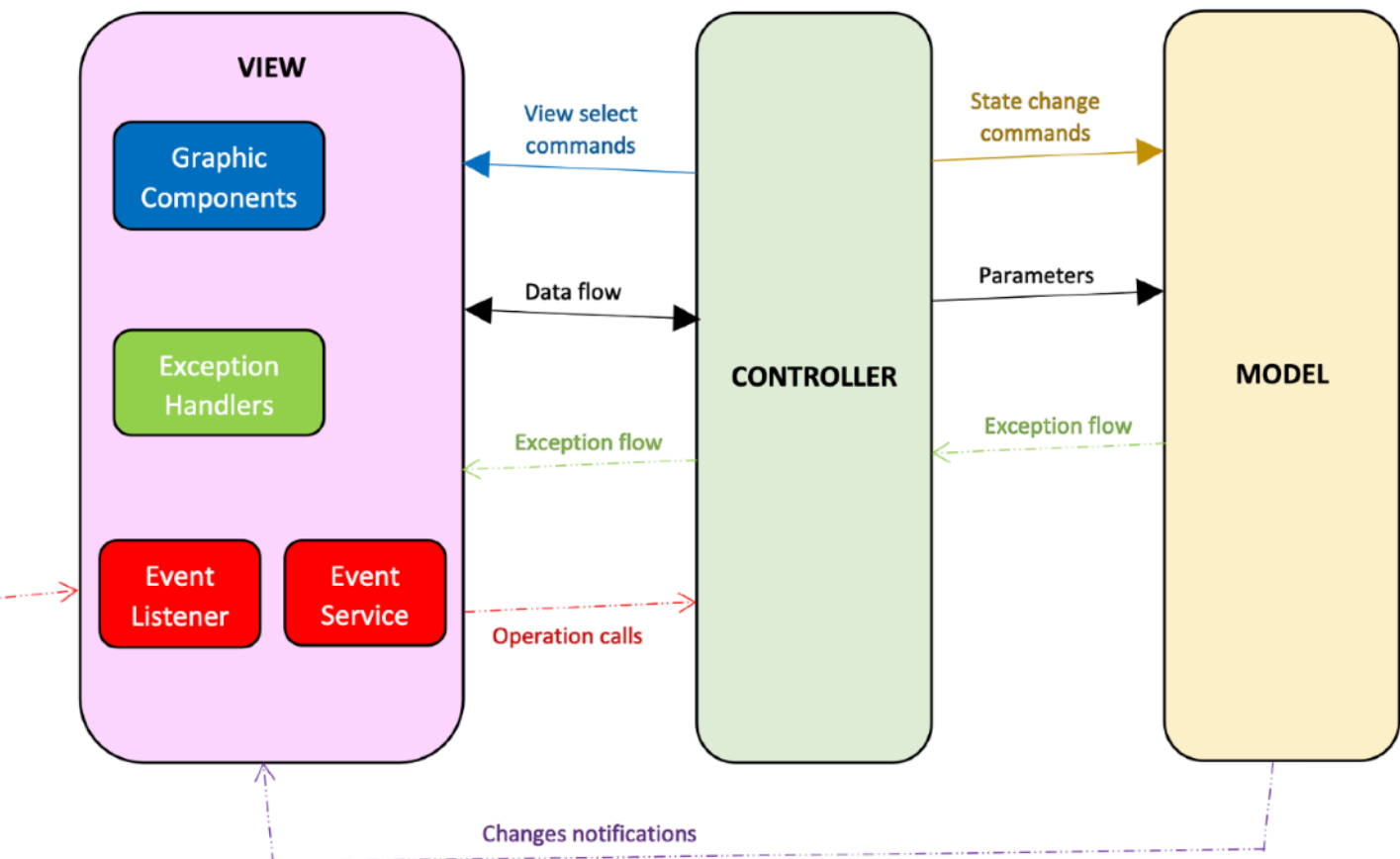
Esempio: Un'informazione inserita dall'utente come "12 ottobre 1942" può essere convertita nel formato interno {1942, 10, 12} per essere gestita correttamente dal sistema.

La *View* ha anche il compito di **mantenere aggiornati i dati visualizzati**.

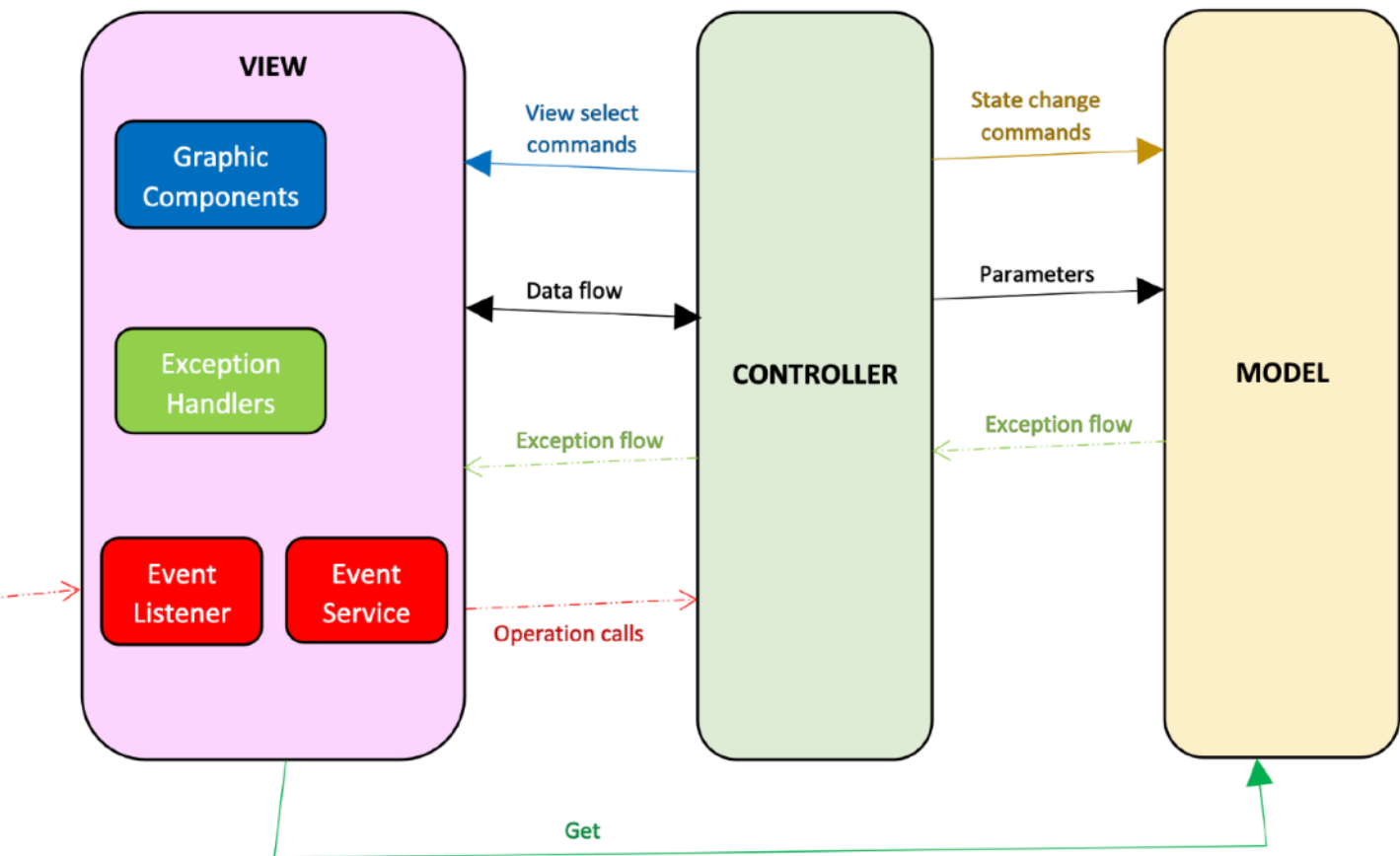
Per farlo, può operare secondo due diverse modalità:

- **Push model:** si basa sull'applicazione del *pattern GoF Observer*. In questo approccio, le *View* si registrano come **osservatori del Model** e ricevono da esso **aggiornamenti automatici in tempo reale** ogni volta che i dati cambiano.
- **Pull model:** viene impiegato quando le *View* (*controller grafico view*) preferiscono **richiedere esplicitamente gli aggiornamenti al Model** in momenti specifici, anziché riceverli automaticamente in tempo reale.

Push model:



Pull model:



La *Boundary* si occupava di **definire le funzionalità** del sistema.

La *View*, invece, si concentra su **come presentare i dati** necessari per consentire l'esecuzione di tali operazioni.

In altre parole, le *View* sono **responsabili della logica di presentazione** dei dati.

BOUNDARY (BCE) VS VIEW (MVC)

Le **Boundary** appartengono alla **fase di analisi** (schema BCE) e si occupano di **definire le funzionalità** del sistema, ovvero *quali operazioni* devono essere messe a disposizione dell'utente.

Le **View**, invece, appartengono alla fase di progettazione (schema MVC) e si concentrano su *come* tali funzionalità vengono presentate e rese accessibili graficamente.

Non è detto che la view sia una classe!!!!!!

CONTROLLER

Nel pattern MVC, il *Controller* è l'elemento che gestisce le azioni dell'utente e coordina il dialogo tra la *View* (interfaccia) e il *Model* (logica/dati).

Tuttavia, in un sistema ben progettato, il "Controller" si divide in due ruoli distinti:

1. **Controller grafico** → vicino alla *View*, coordina l'interazione tra View ed il Controller Applicativo (o il Model in sola lettura) per la realizzazione di una funzionalità. Un controller per le parti di presentazione.

I suoi compiti principali:

- **Coordina l'interazione tra la View e il Controller applicativo**
Quando l'utente preme un pulsante o inserisce dati, il controller grafico raccoglie l'input, lo elabora e chiama il controller applicativo (oppure il *Model* in sola lettura).
- **Converte i formati dei dati**
Se la *View* non lo fa, si occupa di tradurre i dati dal **formato esterno** (testo, numeri, date inserite dall'utente) al **formato interno** (oggetti o strutture dati del Model) e viceversa.
Esempio: da "12/10/2025" → {2025, 10, 12}.
- **Realizza la "mappa" tra input dell'utente e processi del Model**
Decide *quale azione del Model* eseguire in risposta a *quale evento dell'utente*.
Esempio: clic su "Login" → chiamata a `controllerApplicativo.autenticaUtente()`.
- **Crea o seleziona le View richieste**
Se un'operazione richiede un cambio di schermata, il controller grafico si occupa di mostrare la View corretta (per esempio, aprire una nuova finestra o passare a un'altra JSP).

2. **Controller applicativo** → vicino al *Model*, gestisce la logica dell'applicazione vera e propria. Raffina il concetto di controller in BCE.

Le sue caratteristiche principali:

- **Raffina il concetto di Controller del BCE**
Ricordate che nel *BCE* avevamo le classi *Control* che gestivano i casi d'uso?

Il controller applicativo è la versione più concreta e ingegnerizzata di quel concetto.

- **Gestisce una specifica azione dell'utente**
Prende la richiesta arrivata dal controller grafico (es. “conferma prenotazione”, “effettua login”) e la traduce in **una o più operazioni sul Model** (es. PrenotazioneDAO.salva(), Utente.verificaCredenziali()).

COMPORTAMENTO STATELESS

Un controller applicativo *stateless* significa che non conserva informazioni tra una richiesta e l'altra.

- Due invocazioni identiche, con gli stessi parametri, devono dare **lo stesso risultato**, indipendentemente da chi le chiama o da quando vengono eseguite.
- Ogni richiesta è **autonoma**: se un utente esegue due azioni distinte, vengono create **due istanze diverse del controller applicativo**.

Vantaggio: questo approccio **semplifica la gestione e il testing**, e rende il sistema **scalabile** (più facile da distribuire su più server).

De Angelis distingue due strategie di progettazione:

Controller grafico e applicativo uniti (bundle unico).
Controller grafico e applicativo separati (disaccoppiati).

Bundle unico

Il controller grafico **contiene direttamente** la logica di controllo.

Il controller grafico **fa tutto**: gestisce l'interfaccia e le operazioni del Model. È una soluzione più semplice da realizzare, ma meno modulare. Se cambia la View (GUI o pagina web), devi modificare anche il controller: **forte accoppiamento**.

Esempio da tatuare: se il controller di una finestra contiene anche la logica per scrivere sul database = questo è un *bundle unico*.

Disaccoppiati

Il controller grafico **collabora** con un controller applicativo distinto.

Il controller grafico si occupa solo dell'interfaccia (raccoglie input, mostra dati).
Chiama metodi del controller applicativo che contiene la logica vera e propria.

Se la View cambia, modifichi solo il controller grafico.

Il *controller applicativo* resta invariato: **maggior riuso e manutenibilità**.

Esempio da tatuare:

In un sistema MVC web:

- LoginServlet (controller grafico) → riceve username/password.
- LoginController (controller applicativo) → gestisce l'autenticazione usando il *Model*.

In questo corso, il Prof. De Angelis adotta la seconda opzione: controller grafico e applicativo disaccoppiati.

Il De Angelis lo chiede all'esame: “Perché scegliamo l'approccio disaccoppiato”?

Risposta possibile:

Professore, scegliamo l'approccio disaccoppiato perché consente di separare in modo netto la logica di presentazione dalla logica di controllo dell'applicazione, migliorando così la manutenibilità, il riuso e la scalabilità del sistema.

In questo modo:

- Il controller grafico si occupa solo dell'interazione con l'utente e della gestione della View (GUI, JSP, ecc.);
- Il controller applicativo, invece, realizza la logica vera e propria dei casi d'uso, interagendo con il Model.

Se la View evolve (ad esempio, se cambiamo interfaccia o framework grafico), le modifiche non si propagano alla logica applicativa, ma rimangono confinate nel controller grafico.

Questo riduce l'accoppiamento tra i componenti e semplifica l'evoluzione del software nel tempo.

Inoltre, un controller applicativo disaccoppiato è più facilmente testabile, può essere riutilizzato in contesti diversi (ad esempio GUI, CLI o Web), e mantiene un comportamento stateless, come richiesto dai principi di buona progettazione.

MODEL

Costituisce la rete di entità partecipanti alla logica di applicazione.

Esempio:

In un sistema di prenotazioni, il Model comprende entità come **Utente**,
Prenotazione, **Evento**, **Pagamento**.

Sono queste classi che descrivono **cosa il sistema fa e come i dati interagiscono tra loro**.

Un buon modello di entità non dovrebbe esporre lo stato di un oggetto.

Questo è un principio chiave della programmazione orientata agli oggetti:

le entità devono proteggere i propri dati interni e offrire solo operazioni significative per interagire con essi.

In pratica:

Il Model non deve permettere a chiunque di modificare i suoi dati, ma deve fornire metodi controllati per farlo.

Questo serve per mantenere la coerenza e l'integrità dei dati.

Esempio:

Meglio offrire `aggiungiPrenotazione()` che permettere di accedere direttamente alla lista delle prenotazioni e modificarla manualmente.

Espone **metodi pubblici (operazioni)** che rappresentano ciò che quell'entità può fare (es. `creaPrenotazione()`, `calcolaTotale()`, `verificaDisponibilità()`);

Il Model del pattern MVC discende direttamente dal concetto di Entity nel BCE: raffinamento della entity.

Infatti, sia nel BCE sia nell'MVC:

- l'entità **incapsula il proprio stato**, cioè i suoi **attributi privati**;
- e offre **metodi pubblici** per modificarlo o leggerlo.

Connessione dinamica tra il Model e la View.

Quando il Model cambia (perché il Controller gli ha chiesto di modificare un dato), deve **informare la View** per aggiornare l'interfaccia.

Il Controller non può aggiornare manualmente tutte le View ogni volta, quindi la responsabilità di avvisarle può spettare al Model.

Questo è esattamente ciò che fa il **pattern Observer**:

La View si “iscrive” come osservatore del Model = Subject, e quando lo stato cambia, riceve una notifica per aggiornarsi (PUSH MODEL).

Ovviamente il model notificherà il controller grafico nei nostri progetti, il controller della view, non è che notifica proprio la view.

Ad esempio: In una GUI, se il Model aggiorna la temperatura letta da un sensore, la View (che mostra il numero sullo schermo) viene avvisata e aggiorna il valore automaticamente.

Esempio 2:

Il Controller dice: `model.aggiornaPrezzo(20.0)`.

- Il Model aggiorna il suo attributo e poi “notifica” le View collegate.
- Le View, appena ricevono la notifica, si aggiornano (es. la GUI mostra il nuovo prezzo).

COSA NON DEVE SUCCEDERE

Il Model **non deve mai conoscere** dettagli concreti della View (GUI, JSP, FXML, CLI ecc.). Se il Model chiama direttamente un metodo della View (`view.aggiorna()`), il sistema diventa troppo accoppiato e non più manutenibile.

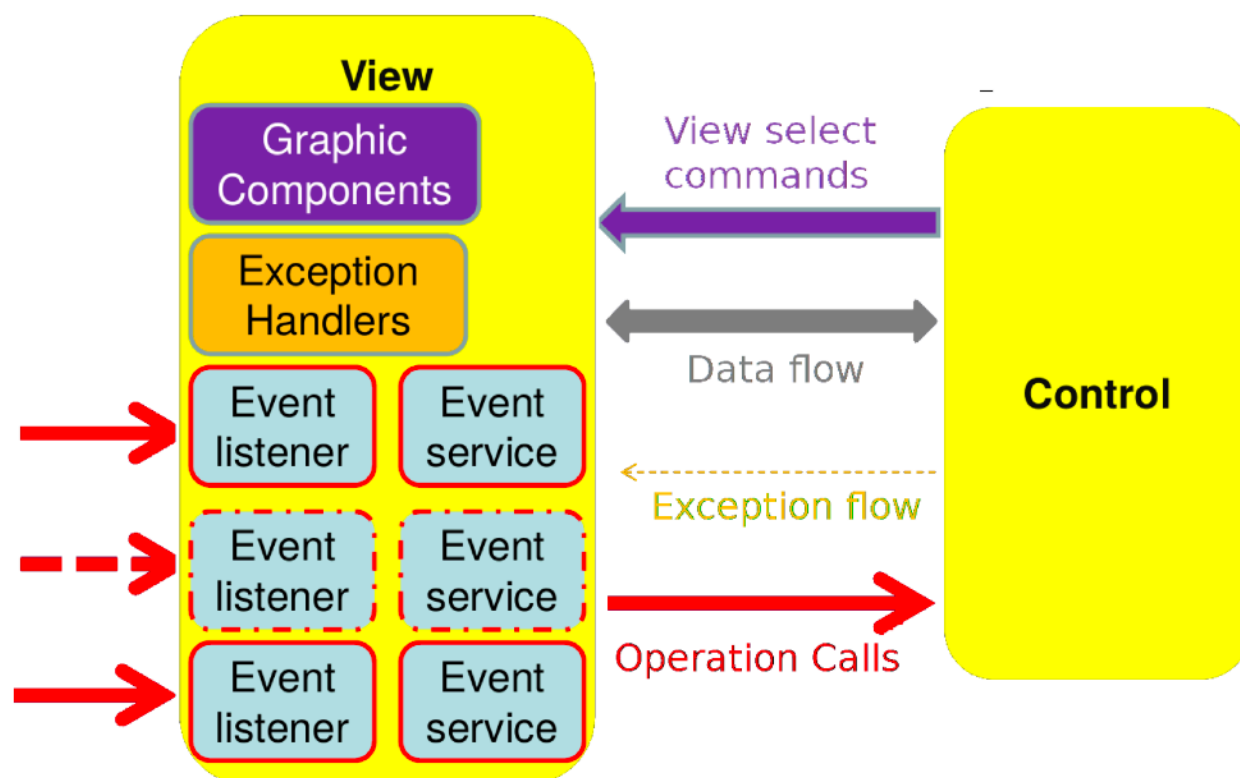
Il Controller non deve aggiornare i dati “a mano” nella View: deve solo orchestrare il flusso.

Il Model non deve “parlare” esplicitamente con la View.

Al massimo emette un evento o una notifica (“sono cambiato”), e la View si aggiorna da sola se registrata come osservatore.

Attenzione: quanto sto scrivendo in queste righe rappresenta la parte più importante dell'intero corso di Ingegneria del Software e Progettazione Web (ISPW), limitatamente alla sezione tenuta dal professor De Angelis.

MVC – INTERAZIONE UTENTE



L'immagine mostra che la **View** non è solo grafica, ma contiene diversi **sottocomponenti** che:

- ascoltano gli eventi dell'utente,
- li traducono in comandi,
- li inoltrano al **Control** (cioè al controller applicativo).

Il **Control** poi esegue la logica dell'applicazione e può restituire dati o segnalare eccezioni alla View.

ELEMENTI DELLA VIEW

Graphic Components

Rappresentano l'**interfaccia utente vera e propria**: finestre, pulsanti, form HTML, caselle di testo, tabelle. Sono gli **elementi visibili** con cui l'utente interagisce.

Exception Handlers

Si occupano di **gestire errori o eccezioni** che possono verificarsi durante l'interazione con la GUI (es. input non valido). Impediscono che errori a livello di View si propaghino verso il Control. (es. "Credenziali non valide", "Campo obbligatorio mancante").

Non gestiscono la logica dell'errore applicativo — gestiscono solo gli errori di presentazione o di interazione grafica.

La **logica vera e propria dell'errore** (cioè cosa succede quando qualcosa va storto) è responsabilità del **Controller (grafico o applicativo)**, che **decide come reagire e quale vista mostrare in seguito**.

Event Listener

Sono gli "**ascoltatori di eventi**", cioè oggetti che rimangono in attesa di un'azione dell'utente: clic di un pulsante, digitazione, selezione da menu, invio di un form, ecc.

Quando l'utente agisce, l'**event listener** cattura l'evento e lo passa all'**event service** corrispondente.



Event Service

È la parte della View che traduce l'evento in una chiamata a un'operazione del Controller (cioè un "operation call").

Nell'architettura MVC disaccoppiata del prof. De Angelis, "l'Event Service" della View passa sempre prima dal **Controller grafico**, che funge da intermediario tra l'interfaccia e il Controller applicativo. Quest'ultimo, a sua volta, si occupa della logica di business e dell'interazione con il Model.

Esempio pratico (app web): L'utente invia un form di login. La View (JSP) → genera l'evento → Servlet (Controller grafico). La Servlet raccoglie username e password, li converte in stringhe pulite. La Servlet chiama il **LoginController** (Controller

applicativo). Quest'ultimo usa il **Model** (**UtenteDAO**) per verificare le credenziali. La Servlet (grafico) decide poi quale JSP mostrare (**View**).

Quindi, a coronamento di tutto ciò mi sento responsabile di asserire: **gli Event Service nella View invocano le operazioni del Controller applicativo (Control).**

Quindi **View** → **Control** si passa.

Flusso **Control** → **View**, in cui il controller invia i risultati (es. elenco prenotazioni, dati utente, messaggi).

In base al risultato ottenuto, il **Controller** può dire alla View *quale schermata mostrare*.

Esempio: dopo un login corretto, il Control dice “mostra la homepage”; se fallisce, “mostra la pagina di errore”.

Indovina? Quale **Controller**? No, non è il **controller applicativo** a dire direttamente alla View quale schermata mostrare. È il **controller grafico** a decidere **quale View** deve essere visualizzata in base al risultato restituito dal controller applicativo.

Il Controller applicativo si occupa della logica di controllo dell'applicazione, cioè dei casi d'uso. Prende in ingresso dati “puliti” (già validati) dal controller grafico. Interagisce con il **Model** per eseguire le operazioni necessarie. Restituisce un risultato logico, ad esempio: “autenticazione riuscita”, “utente non valido”, “errore di connessione” oppure un oggetto dati da mostrare. Ma **non sa nulla della View**: non decide come il risultato sarà mostrato.

Il suo unico compito è dire “il caso d'uso è andato bene o male”.

Il controller applicativo esegue la logica del caso d'uso e restituisce un risultato, che il controller grafico interpreta per scegliere la vista più appropriata. In questo modo il controller applicativo rimane indipendente dalla parte grafica, ed è proprio quello che vuole il **Prof. De Angelis**.

Ricorda, lo ripeto all'infinito.

Nel pattern MVC disaccoppiato (come lo insegna il prof. De Angelis):

- Il **Controller grafico** e la **View** vogliono sapere quando il **Model cambia**, per aggiornare la GUI, ricaricare dati, ecc.
- Ma il **Model non deve sapere chi lo sta visualizzando**, né come.

Soluzione: si applica il **pattern Observer (GoF)**.

Elemento	Ruolo	Relazione
Model	Observable (Subject)	Notifica i cambiamenti
Controller grafico / View	Observer	Si aggiornano quando il Model cambia
Controller applicativo	No Observer	Interagisce solo in scrittura con il Model

Componente	Responsabilità	Comunicazione diretta con
View	Mostra dati e riceve input	Controller grafico
Controller grafico	Converte input, invoca logica applicativa, seleziona la View	Controller applicativo
Controller applicativo	Gestisce i casi d'uso, coordina il Model	Model
Model	Mantiene dati e logica, notifica le View/Controller grafici	Observer (View o Controller grafico)

La View cattura un evento dell'utente (es. click, invio form)

Lo inoltra al Controller grafico

Il Controller grafico traduce i dati in formato interno e chiama il Controller applicativo

Il Controller applicativo interagisce col Model (legge o scrive dati)

Se il Model cambia, notifica View/Controller grafico (tramite pattern *Observer*)

Il Controller grafico aggiorna o seleziona la View da mostrare.

Il controller applicativo non parla direttamente con la View: tutto passa sempre per il **controller grafico**, che funge da *ponte* tra logica e interfaccia.

Il **controller applicativo** *non deve sapere nulla* della View o del controller grafico. Si limita a restituire un risultato, o a sollevare eccezioni controllate, o a restituire oggetti di dominio. Sarà poi il **controller grafico** a interpretare quel risultato e a decidere *come e cosa mostrare*.

ESEMPIO DI CONTROLLER GRAFICO - SERVLET JAVA

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    private final LoginController controllerApplicativo = new LoginController();

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // chiama il controller applicativo @Simone Remoli
        LoginResult risultato = controllerApplicativo.autentica(username,
password);

        // interpreta il risultato e decide cosa mostrare
        if (risultato.isOk()) {
            request.setAttribute("utente", risultato.getUtente());
            request.getRequestDispatcher("home.jsp").forward(request, response);
        } else {
            request.setAttribute("errore", risultato.getMessaggioErrore());
            request.getRequestDispatcher("login.jsp").forward(request,
response);
        }
    }
}
```

ESEMPIO DI CONTROLLER APPLICATIVO

```
public class LoginController {

    public LoginResult autentica(String username, String
password) {
        Utente utente = UtenteDAO.findByUsername(username);

        if (utente != null &&
utente.verificaPassword(password)) {
            return new LoginResult(true, utente, null);
        } else {
            return new LoginResult(false, null, "Credenziali
non valide");
        }
    }
}
```

```
}
```

Il **LoginResult** è un oggetto di servizio (*DTO*) che trasporta l'esito di un'operazione tra il controller applicativo e quello grafico. Quindi **non è un Model**, ma fa parte della **logica di controllo**.

È il **Controller applicativo** che chiama i DAO (non DTO, volevo proprio scrivere DAO) per ottenere o aggiornare Model, ma non accede mai direttamente al DB.

I DAO non sono parte del Model, ma si trovano a un livello inferiore e si occupano di accedere alla base di dati. Tuttavia, i DAO restituiscono oggetti che appartengono al Model, cioè entità di dominio come **Utente** o **Prenotazione**. In questo modo il controller applicativo lavora sempre con oggetti del dominio, non con dati grezzi del database.

Nel modello architetturale di De Angelis,

- il **controller applicativo** è l'unico che *decide* le modifiche al Model,
- ma **non è obbligato** a usare una DAO ogni volta.

La DAO è un **servizio tecnico** che il controller usa **solo quando serve persistenza**.

Il Model invece **incapsula la logica** e può cambiare stato anche solo in memoria.

Il DAO è il “traduttore” tra mondo **relazionale** (DB) e mondo **a oggetti** (Model).

Legge righe dal DB e crea oggetti Model; prende oggetti Model e li salva in tabelle.

```
public class UtenteDAO {  
    public Utente findById(int id) {  
        ResultSet rs = ...  
        if (rs.next()) {  
            return new Utente(  
                rs.getInt("id"),  
                rs.getString("nome"),  
                rs.getString("cognome"),  
                rs.getString("email"),  
                rs.getString("password")  
            );  
        }  
        return null;  
    }  
}
```

Qui il DAO prende i **dati del DB** e costruisce un **oggetto del Model** con le stesse informazioni, ma strutturate in modo orientato agli oggetti.

(Chi ha seguito il corso di Basi di Dati del Prof. Pellegrini non ha problemi di comprensione).

REGOLA IMPORTANTE: Il **Model** non deve essere una copia del database, ma deve rappresentare le **stesse informazioni concettualmente rilevanti** per la logica dell'applicazione. Può contenere più o meno attributi rispetto al DB, purché mantenga la **coerenza semantica** con i dati memorizzati.

Il **Controller applicativo** chiede dati al **Model** (o meglio, alla **DAO** che accede al DB). La **DAO** interroga il database, ricostruisce gli oggetti del **Model** (ad esempio Utente, Prenotazione, Ordine, ecc.). La **DAO** restituisce questi oggetti al **Controller applicativo**. Il **Controller applicativo**: li elabora, oppure li incapsula in un risultato logico (es. ResultObject, DTO), e li restituisce al **Controller grafico**. Il **Controller grafico** li interpreta e li passa alla **View** per la visualizzazione.

Il Controller applicativo parla con il Model; poi **ritorna al Controller grafico** un risultato (che può contenere uno o più oggetti del Model, o un oggetto "Result" come LoginResult); sarà il **Controller grafico** a passarli alla **View**, in un formato adeguato alla presentazione.

Il controller applicativo può restituire oggetti del Model, ma non li passa direttamente alla View. Li restituisce al controller grafico, che si occupa di interpretarli e trasferirli alla View per la visualizzazione. In questo modo si mantiene il **disaccoppiamento** tra logica applicativa e logica di presentazione.

Evoluzione del Model

Un buon modello architetturale deve permettere di evolvere senza rompere tutto il resto.

Supponiamo di cambiare il nome ad una classe nel model.

Qual è l'impatto?

Le conseguenze potrebbero partire dal model ad arrivare fino alla view: se la classe model "Pippo" viene utilizzato come parametro attuale in un'operazione del controller, devo cambiare la definizione di quella chiamata.

Esempio: prima salvavi una data come String, ora come LocalDate; prima avevi un prezzo in int, ora in BigDecimal. Questi cambiamenti impattano il Model e, se il sistema non è ben disaccoppiato, si propagano anche su DAO, Controller e View.

Evoluzione delle View/Boundary

Nel BCE, la Boundary (cioè le classi che interagiscono con l'utente);

Nell'MVC, la View e il Controller grafico.

Significa cambiare il modo in cui l'utente interagisce con il sistema: prima usavi una GUI JavaFX, ora una webapp JSP; prima input da tastiera, ora da touchscreen; prima file CSV, ora API REST.

Se l'architettura è ben fatta (View disaccoppiata dal Model), puoi cambiare la tecnologia d'interfaccia **senza toccare la logica** del programma.

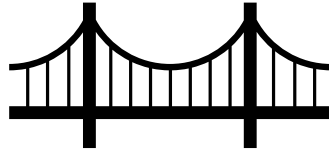
Esempio: puoi rifare la GUI del gestionale, ma il controller applicativo e le classi Model restano uguali.

Ogni evoluzione deve avere impatto solo sul livello corrispondente, non su tutto il sistema.

Tra controller grafico e controller applicativo si scambiano tipicamente oggetti del Model, poiché rappresentano i dati del dominio. Tuttavia, in molti casi è opportuno usare oggetti di servizio o DTO che incapsulano il Model e altre informazioni, così da mantenere il **disaccoppiamento** e migliorare la chiarezza della comunicazione tra i due livelli.

Per mitigare questi due problemi, introduciamo delle **nuove classi** che hanno delle nuove responsabilità, e sono classi che non fanno parte del dominio del problema, in questo corso vengono chiamate **bean**.

CLASSI BEAN



MODELLARE LE CLASSI BEAN FIN DAL BCE, PER POI RAFFINARLE IN MVC: durante l'analisi BCE (Boundary-Control-Entity), devi già individuare i flussi di dati tra boundary e control, e quindi capire quali Bean serviranno.

In sostanza: i Bean nascono nell'analisi (BCE) e maturano nella progettazione (MVC).

Un Bean è un contenitore di informazioni, con campi privati e accesso controllato tramite metodi pubblici.

Praticamente sono un elemento di buffering per la comunicazione fra classi Boundary, Control ed Entity.

Le classi Bean fanno da ponte (o “zona cuscinetto”) tra la Boundary = View + Controller grafico in MVC (cioè la parte che interagisce con l'utente, es. JSP, GUI, servlet), e il resto del sistema (Control ed Entity).

In pratica: la Boundary non parla direttamente con le Entity, ma passa e riceve dati tramite i Bean.

Principalmente sono interposte fra le classi **Boundary** e il resto delle classi del VOPC.

Significa che le classi Bean si trovano **tra View/Boundary e Control/Entity**, cioè esattamente nel punto in cui avviene lo **scambio dei dati** (input/output).

Le **Boundary** (interfacce grafiche o JSP) **non devono conoscere** le Entity interne. Il Bean serve a *trasportare i dati* tra i due livelli senza esporre direttamente le Entity.

I Bean possono controllare che i dati inseriti siano formalmente corretti (prima ancora che arrivino alla logica applicativa), **se non l'ho fatto nel controller grafico: ad esempio controllo se la data di nascita è valida la può fare StudentBean.java con una funzione privata dentro di essa.**

Sostanzialmente la boundary (View + Controller grafico) non può invocare il controller applicativo passandogli dei model, ma delle bean.

View + controller grafico comunica con il controller applicativo tramite l'ausilio delle bean, che non sono più dei model.

“Quindi, le operazioni che ci sono nel controller applicativo, sono operazioni che prendono delle classi bean. Non possono prendere delle entity.”

-Guglielmo De Angelis

Cosa succede in pratica

1. L'utente inserisce dati nella View

```
<form action="CreaUtenteServlet" method="post">
```

```
Nome: <input type="text" name="nome">
```

```
Email: <input type="text" name="email">
```

```
</form>
```

2. Il Controller grafico raccoglie questi dati.

```
UtenteBean bean = new UtenteBean();  
bean.setNome(request.getParameter("nome"));  
bean.setEmail(request.getParameter("email"));
```

Il Controller grafico (es. una servlet) prende i dati della View e li inserisce in un **Bean**. Qui la servlet non crea direttamente un oggetto **Utente** (Model), ma usa una **Bean**, che serve da contenitore temporaneo dei dati di input.

3. Il Controller grafico passa la Bean al Controller applicativo.

```
UtenteController controllerApp = new UtenteController();
controllerApp.creaNuovoUtente(bean);
```

4. Il Controller applicativo traduce la Bean in Model

```
public void creaNuovoUtente(UtenteBean bean) {
    // converto la bean nel model
    Utente u = new Utente();
    u.setNome(bean.getNome());
    u.setEmail(bean.getEmail());

    utenteDAO.save(u);
}
```

Il **Controller applicativo** si occupa di trasformare i dati "grezzi" (contenuti nella Bean) in veri e propri **oggetti Model**, pronti per essere salvati o elaborati.

Struttura tipica di una classe Bean

Sono classi Java semplici, senza dipendenze da framework particolari, senza ereditarietà complessa, e con costruttore vuoto.

I dati vengono memorizzati come **variabili private**, accessibili solo tramite getter/setter. Se il Bean deve validare i dati in input, i controlli sono **metodi privati interni**. Eventualmente possiede metodi di controllo sui dati.

Le Bean non sono copie del Model, ma contengono solo i dati necessari al caso d'uso.

Possono includere un sottoinsieme degli attributi del Model, oppure dati derivati o aggregati.

Servono a **disaccoppiare la View e il Controller grafico** dalla struttura interna del Model, garantendo sicurezza, modularità e flessibilità.

Le Bean non devono contenere necessariamente le stesse informazioni del Model.

Il Model rappresenta la logica e lo stato del dominio, mentre le Bean servono come contenitori di dati per la comunicazione tra View, Controller grafico e Controller applicativo. Contengono quindi solo le informazioni rilevanti per un determinato caso d'uso, eventualmente arricchite con dati di servizio o derivate.

Le Bean non servono solo in input

Finora le stiamo vedendo come contenitori per i dati inseriti dall'utente.

Le Bean non servono solo a ricevere dati, ma anche a **presentarli**.

Il **Controller applicativo** costruisce una Bean, la **riempie con i dati del Model**, la passa al **Controller grafico**, che a sua volta la inoltra alla **View**.

Esempio.

Immagina un caso d'uso "visualizza prenotazione":

Controller applicativo.

```
public class PrenotazioneControllerApplicativo {  
  
    public PrenotazioneBean getDettagliPrenotazione(int id) {  
        Prenotazione p = prenotazioneDAO.findById(id); //  
Model  
  
        // Creo una bean di output per restituire i dati  
        PrenotazioneBean bean = new PrenotazioneBean();  
        bean.setNomeCliente(p.getUtente().getNome());  
        bean.setData(p.getData().toString());  
        bean.setPrezzo(p.getPrezzo());  
        bean.setStato(p.getStato());  
  
        return bean;  
    }  
}
```

Controller grafico (Servlet).

```
@WebServlet("/dettagliPrenotazione")  
public class DettagliPrenotazioneServlet extends HttpServlet  
{  
    private PrenotazioneControllerApplicativo controller =  
new PrenotazioneControllerApplicativo();  
}
```

```

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        int id =
Integer.parseInt(request.getParameter("id"));
        PrenotazioneBean bean =
controller.getDettagliPrenotazione(id); // riceve la bean di
output

        request.setAttribute("prenotazione", bean);

request.getRequestDispatcher("dettagliPrenotazione.jsp").forw
ard(request, response);
    }
}

```

View (JSP)

```

<%@ page import="model.bean.PrenotazioneBean" %>
<%
    PrenotazioneBean p = (PrenotazioneBean)
request.getAttribute("prenotazione");
%>

<h2>Dettagli prenotazione</h2>
<p>Cliente: <%= p.getNomeCliente() %></p>
<p>Data: <%= p.getData() %></p>
<p>Prezzo: <%= p.getPrezzo() %> €</p>
<p>Stato: <%= p.getStato() %></p>

```

Quindi: le Bean possono essere di due tipi:

Tipo di Bean	Direzione	Ruolo
Bean di input	View → Controller applicativo	Raccolgono e validano i dati dell'utente
Bean di output	Controller applicativo → View	Contengono i dati del Model pronti per la presentazione

Quindi, il primo passo che dovrà fare il controller è recuperare le informazioni

della entity associata (o le entity associate) al bean passatogli dal controller grafico.

Posso avere una **StudenteBEAN** e un **ModelStudente**.

Il model/entity avrà un nome, codicefiscale e una matricola.

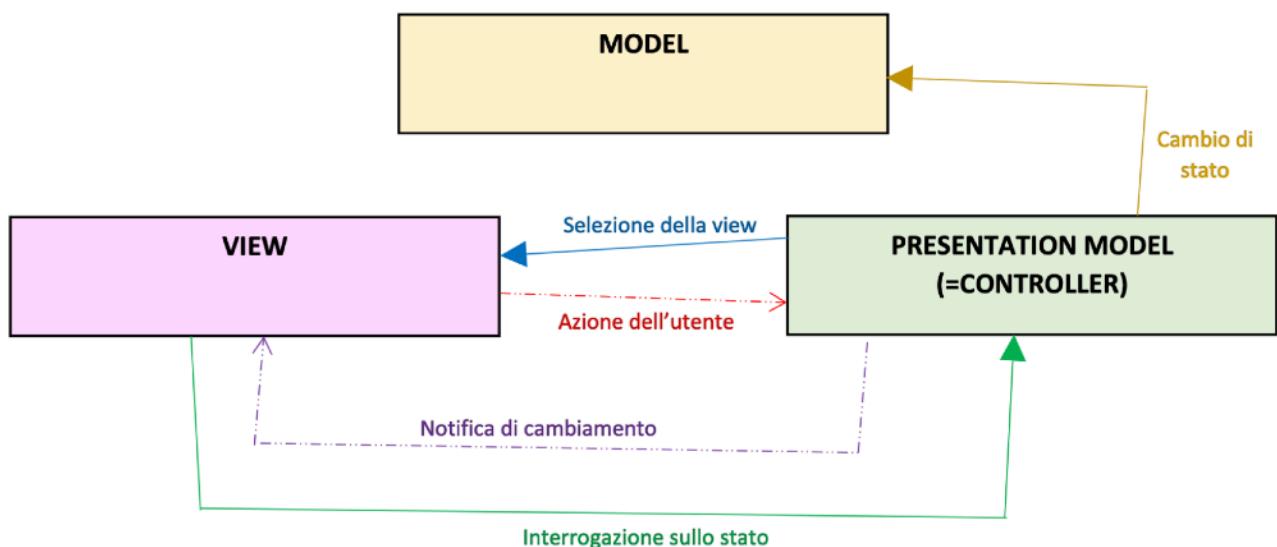
Il Bean avrà tutti getter e setter e avrà come attributi studentID, name, e forse la matricola.

Il controller grafico passerà una bean studente al controller applicativo e quest'ultimo trasformerà il bean in un'istanza di tipo studente.

Il bean può anche avere un unico attributo che si chiama S che è di tipo studente.

MVP - ANALISI IN DETTAGLIO

L'MVP funziona praticamente identico all'MVC.



L'unica variante è che la View non parla più direttamente con il Model, ma comunica solo con il Presenter (o “Presentation Model”).

Elemento	Ruolo
View	Mostra i dati e inoltra le azioni dell'utente al Presenter.
Presentation Model (= Controller)	Riceve l'azione dell'utente, aggiorna il Model e poi aggiorna la View.
Model	Contiene i dati e la logica di dominio.

A differenza della view nel modello MVC:

- non può accedere direttamente al model per acquisire i dati da visualizzare;
- deve sempre interagire con il controller (presentation model), anche per richiamare i metodi *getter* e *setter* che servono a leggere o aggiornare i dati.

Per il resto, la view mantiene le stesse responsabilità di quella del modello MVC, ossia la gestione della logica di presentazione e dell'interfaccia grafica.

Inoltre, anche nel modello MVP, i dati scambiati tra la view e il presentation model devono essere gestiti tramite classi Bean di appoggio, per evitare un accoppiamento diretto tra la view e il model.

Infine, il presentation model e il model nel pattern MVP sono del tutto analoghi rispettivamente al controller e al model del pattern MVC.

Ricordiamo prima succedeva che la view interagiva con il controller applicativo, quest'ultimo inviava dei risultati e notificava lo stato del model, e la view se voleva poteva accedere in modalità push (o pull) al model.

Nell'MVP questo è impossibile, tutto qui.

Esempio. Supponiamo di avere questo model.

```
// MODEL
public class CalcolatriceModel {
    private int numero;
```



```

    public void setNumero(int numero) {
        this.numero = numero;
    }

    public int calcolaDoppio() {
        return numero * 2;
    }
}

```

Il Model contiene la logica (in questo caso: raddoppiare un numero).

La View non lo conosce direttamente — potrà accedervi solo tramite il *Presentation Model*.

La bean di appoggio è la seguente:

```

// BEAN
public class CalcolatriceBean {
    private int numeroInput;
    private int risultato;

    public int getNumeroInput() { return numeroInput; }
    public void setNumeroInput(int numeroInput)
{ this.numeroInput = numeroInput; }

    public int getRisultato() { return risultato; }
    public void setRisultato(int risultato) { this.risultato
= risultato; }
}

```

Il Presentation Model (Controller) è il seguente:

```

// PRESENTATION MODEL
public class CalcolatricePresenter {
    private CalcolatriceModel model;

    public CalcolatricePresenter() {
        this.model = new CalcolatriceModel();
    }

    // Metodo invocato dalla View
    public void calcola(CalcolatriceBean bean) {
        model.setNumero(bean.getNumeroInput());
        int risultato = model.calcolaDoppio();
        bean.setRisultato(risultato);
    }
}

```

La view è la seguente:

```
// VIEW
import java.util.Scanner;

public class CalcolatriceView {
    private CalcolatricePresenter presenter;
    private CalcolatriceBean bean;

    public CalcolatriceView() {
        presenter = new CalcolatricePresenter();
        bean = new CalcolatriceBean();
    }

    public void mostraInterfaccia() {
        Scanner input = new Scanner(System.in);
        System.out.print("Inserisci un numero: ");
        int numero = input.nextInt();

        // La View aggiorna la Bean
        bean.setNumeroInput(numero);

        // La View non parla col Model, ma col Presenter
        presenter.calcola(bean);

        // La View legge il risultato dalla Bean (dati di
output)
        System.out.println("Il doppio è: " +
bean.getRisultato());
    }

    public static void main(String[] args) {
        CalcolatriceView view = new CalcolatriceView();
        view.mostraInterfaccia();
    }
}
```

Immaginando una View console (ma vale anche per una GUI o JSP).

Il Presenter:

- riceve la Bean dalla View,
- usa i dati per aggiornare il Model,

- ottiene il risultato dal Model,
- e lo scrive di nuovo nella Bean (per la View).

MVC in architetture laptop/ stand-alone

Grafica, controller grafico e relazione col controller applicativo	View	↓ ↑ ↑
Classi di controllo	Controller	↓ ↑ ↑
Classi entità e relativa interfaccia verso la persistenza	Model	↓ ↑
Persistenza (database o file system)	Persistency	



Interazioni lettura / scrittura mediate da classi bean



Interazioni lettura / scrittura semplici

In un'architettura stand-alone, tutto il sistema (interfaccia, logica e dati) si trova sullo stesso computer.

Non c'è un server remoto o una rete di comunicazione come nelle architetture client server o web.

Per questo: il pattern MVC resta valido, ma tutti i livelli sono all'interno dello stesso processo applicativo, anche se separati logicamente.

Il controller usa il valore delle bean per recuperare le istanze di model (o queste istanze sono in memoria o in persistenza), ci fa quello che ci deve fare, incapsula il valore di

ritorno nei bean in uscita e viene mostrato.

Le interazioni tra le istanze (MVC) sono tutte in memoria centrale.

MVC IN ARCHITETTURE WEB

Nella pratica, di norma, la view è realizzata con la tecnologia **JSP (Java Server Pages)**, che è una tecnologia di programmazione web in Java che fornisce contenuti dinamici in formato HTML / XML.

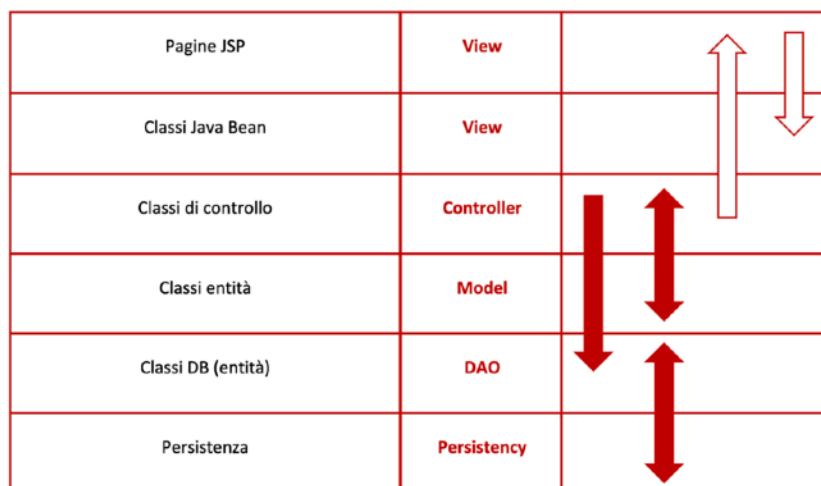
In questo modello, la View è una pagina JSP da caricare o generare, e il Controller non gestisce direttamente l'interfaccia. Per questo entra in gioco un nuovo elemento: le **Servlet**. Le **Servlet** sono il corrispettivo del **controller grafico** nel contesto web.

Le Bean vengono create e gestite dal Controller grafico, cioè dalla Servlet.

La Servlet:

1. legge i dati provenienti dalla View (cioè da un form HTML o da una JSP),
2. crea un oggetto Bean,
3. imposta i valori al suo interno (con `setX()`),
4. passa la Bean al Controller applicativo (che risiede lato server),
5. e infine la rimette a disposizione della JSP (View successiva) per la visualizzazione dei risultati.

Dopo l'elaborazione, la Servlet reinserisce la Bean nella request, così la **View JSP** può leggerla e mostrare i risultati.



Al contrario delle architetture stand-alone, il controller non è responsabile dell'identificazione della view da presentare all'utente come risposta della sua interazione. Per questo motivo, si introducono le **servlet**, che costituiscono uno strumento server-side che ha la responsabilità di istanziare il controller applicativo e, se non se ne occupa la tecnologia JSP stessa, ha anche il compito di selezionare la view successiva da mostrare all'utente.

Fine.